



THE UNIVERSITY OF
SYDNEY

Room Number _____

Seat Number _____

Student Number _____

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

CONFIDENTIAL EXAM PAPER

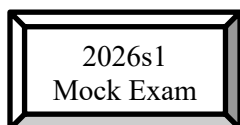
This paper is not to be removed from the exam venue

Computer Science

EXAMINATION

Semester 1 – Final Mock Exam, 2026

COMP9001 Introduction to Programming



For Examiner Use Only

EXAM WRITING TIME: 2 hours
READING TIME: 10 minutes

EXAM CONDITIONS:

This is a RESTRICTED OPEN book exam - specified materials permitted

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Calculator - non-programmable

Formula sheet (provided in the exam paper by unit coordinator)

One A4 sheet of handwritten notes double-sided

MATERIALS TO BE SUPPLIED TO STUDENTS:

Blank scratch paper (1 sheet; more may be provided upon student request)

INSTRUCTIONS TO STUDENTS:

Q	Mark
1	
2	
3	
4	
5	
6	
7	
8	
9	

Do not turn over this page until you are told!

Total _____

- This paper contains nine (9) questions with a total of 100 marks. Answer ALL questions.
 - Questions 1–4 (**Essentials** Section): 40 marks
 - Questions 5–9 (**Advanced** Section): 60 marks
- Please read all the questions first before you begin writing.
- Answer all questions in the spaces provided in this question booklet. Do not write your answers in the answer booklets, which are only to be used for drafting and will not be graded.
- Return this question booklet and any additional answer booklets or materials.
- Write your final answers in black or blue ink, not pencil. Take care to write clearly and legibly. Drawing diagrams using pencil is permitted.
- Please write your student number in the space provided at the top of every odd numbered page of this question booklet.
- State clearly in spaces below the question if the answer is continued on the extra working pages in this question booklet.
- Do not remove any tear or remove any pages from the booklet. Staff are not responsible for missing pages in this case. If you find there are missing pages, please alert the examination supervisors.
- You may refer to the formula sheet for any relevant reference material.

Please tick the box to confirm that your examination paper is complete. ☐

_____ This page was left blank intentionally. Do not write on it. _____

Question 1 (18 marks):

/18

1.a:

/4

Mark the following statements true or false:

- (a) **TRUE** In Python, a tuple is an immutable data type, meaning its elements cannot be changed after creation.
- (b) **TRUE** The expression `10%3` evaluates to 1.
- (c) **TRUE** In Python, a list can contain elements of different data types.
- (d) **FALSE** The boolean expression `not True` evaluates to True.
- (e) **TRUE** A while loop in Python requires a condition that eventually becomes False to terminate.
- (f) **FALSE** The string method `.upper()` modifies the original string in place.
- (g) **TRUE** In Python, `len('hello')` returns 5.
- (h) **TRUE** A function in Python can return more than one value.

1.b:

/1

What is the value of `x` after executing `x=int(9/2)`?

Put your answer here:

4

1.c:

/1

What is the value of `result` after the following code is executed?

```
1 a = (5 + 3) * 2
2 b = a > 10 and a < 20
3 result = (not b) or (a==15)
```

Put your answer here:

False

1.d:

/1

Create a variable named `colours` and assign it a list containing the numbers 'red', 'green', and 'black'.**colours = ['red', 'green', 'black']**

1.e:

What is the output after the following code is executed?

```
1 score = 85
2 if score >= 90:
3     print("HD")
4 elif score >= 75:
5     print("D")
6 elif score >= 65:
7     print("C")
8 else:
9     print("P")
10 print("Done")
```

D
Done

1.f:

/1

Given a name (string) and age (integer), output a message in the following format:

My name is <name> and I am <age> years old.

As an example, a name of 'Abd' and age of 26 outputs:

My name is Abd and I am 26 years old.

Complete the following program (you can assume both inputs are valid):

```
name = input()
```

```
age = int(input())
```

```
print(f'My name is {name} and I am {str(age)} years old.')
```

1.g:

/1

Given a dictionary named `inventory`, remove the key 'banana' from the dictionary.

Note: You must update the existing `inventory` dictionary, not create a new one.

Complete the following program:

```
inventory = {'apple': 10, 'banana': 5, 'orange': 8}
```

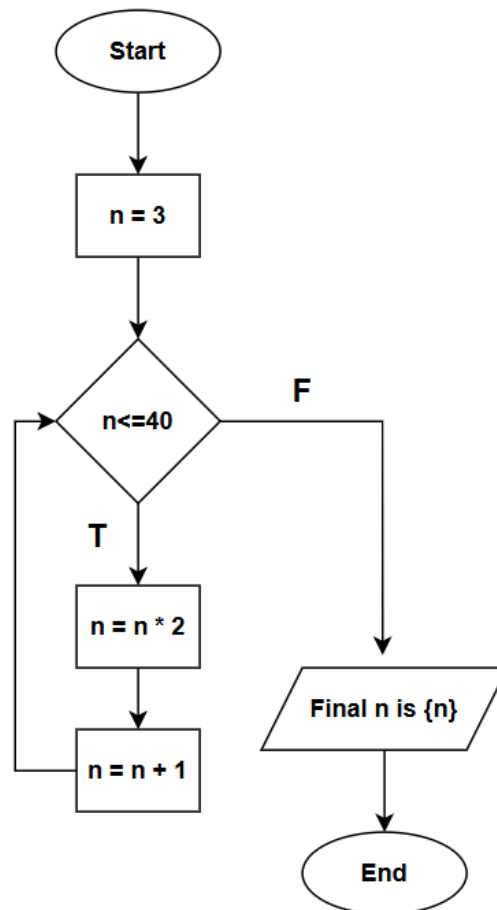
```
inventory.pop('banana')
```

1.h:

/4

Draw a flowchart that represents the control flow of the following code:

```
1  n = 3
2  while n <= 40:
3      n = n * 2
4      n = n + 1
5  print('Final n is ' + str(n))
```



1.i:

/2

Given the following code (note that $0 \% 2$ is equal to 0):

```
1 total = 1
2
3 for k in range(4):
4     if k % 2 == 0:
5         total = total * 3
6     else:
7         total = total + 4
```

How many times will the loop iterate?

4

What will be the final value of `total` after the loop is complete?

25

1.j:

/2

Given the following program:

```
1 def double_elements(lst):
2     for i in range(len(lst)):
3         lst[i] = lst[i] * 2
4
5 values = [1, 2, 3]
6 double_elements(values)
```

(i) What is the value of `values` after `double_elements(values)` is executed? (1)

values = [2, 4, 6]

(ii) Identify whether a list is a mutable or immutable data type. (1)

Put your answer here:

mutable

Question 2 (6 marks):

/6

We require the program `grade_calculator.py` to read all integer test scores from the command line and print the average score as a float rounded to 2 decimal places. The program will be run like this:

```
$ python3 grade_calculator.py 85 92 78 60 95 73
```

Below is the entire code for `grade_calculator.py`. Find as many *different* errors (syntax, logical, or runtime) as you can.

Note: A single line may contain more than one error.

- Assume that the parameter `args` is intended to be the list of command line arguments.
- For each error you find, state the line number and briefly explain why it is an error (simply listing the line number without an explanation will receive no marks).

```
1
2 def calculate_average(args)
3     total = 0
4     for i range(len(args)+1):
5         total = total + args[i]
6         average = total / len(args)
7     return average
8
9 scores = sys.argv[1:]
10 result = calculate_average(scores)
11 print(result)
12
```

There are at least 6 errors, you need to identify all 6 for full marks.

(a) line: **2** ; error is: **Missing colon at end of def**

(b) line: **4** ; error is: **Missing 'in' keyword**

(c) line: **5** ; error is: **args[i] is a string and needs to be converted**

(d) line: **9** ; error is: **Missing import sys**

(e) line: **4** ; error is: **len(args)+1 will result in index out of bounds error**

(f) line: **11** ; error is: **The result must be rounded**

Question 3 (10 marks):

/10

3.a:

/4

Write a function `starts_with(word, ls)` that:

- Takes two parameters:
 - `word`: a non-empty string.
 - `ls`: a list of single-character strings.
- Returns `True` if the first character of `word` is found in `ls`, otherwise return `False`.

Example usages:

- Calling `starts_with('python', ['p', 'j', 'r'])` returns `True`.
- Calling `starts_with('code', ['x', 'y'])` returns `False`.

Restrictions: You may use the `in` keyword **only** in for loops (e.g., `for element in ls`, `for i in range(n)`, `for i in range(len(ls))`).

You **must not** use `in` to check if a value is inside a list or string (e.g., `if element in ls`).

Doing so will result in **zero marks** for the question.

```
def starts_with(word, ls):
    first = word[0]
    for i in range(len(ls)):
        if ls[i] == first:
            return True
    return False
```


3.b:

/6

Using your `starts_with` function from part (3.a), write a program that:

- Reads a list of characters from command-line arguments (ensure to exclude the filename).
- Repeatedly prompts the user to enter a word, until the user enters 'stop'.
- For each word entered (other than 'stop'):
 - Use the `starts_with` function to check if the word starts with one of the characters provided in the command-line arguments.
 - If it does, add the word to a list called `final_words`.
- After the user enters 'stop', it prints the list of `final_words`.

Note: Use the existing `starts_with` function from part (3.a); no need to rewrite it. If you were unable to implement it, assume it exists.

Example run [user inputs (i.e., words typed by the user) are shown in **bold**]:

```
$ python3 program.py p r s
Word: python
Word: java
Word: ruby
Word: sql
Word: stop
Matches: ['python', 'ruby', 'sql']
```

Hint: You will need to use the `sys` module.

```
import sys

chars = sys.argv[1:]
final_words = []

word = input('Word: ')
while word != 'stop':
    if starts_with(word, chars):
        final_words.append(word)
    word = input('Word: ')

print('Matches:', final_words)
```

Question 4 (6 marks):

/6

Given the following two modules:

bankaccount.py

```
1 class BankAccount:
2     def __init__(self, owner: str, balance: int = 0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount: int):
7         self.balance = self.balance + amount
8
9     def withdraw(self, amount: int):
10        self.balance = self.balance - amount
```

bank_program.py

```
1 from bankaccount import BankAccount
2
3 acc1 = BankAccount("Alice", 100)
4 acc2 = BankAccount("Bob")
5 acc1.deposit(50)
6 acc2.withdraw(20)
7 print(acc1.balance)
8 print(acc2.balance)
```

4.a:

/2

What will be the output produced from lines 7 and 8 in bank_program.py after the program is executed?

Line 7 will output:

150

Line 8 will output:

-20

4.b:

/2

Rewrite the withdraw method so that:

- If a withdrawal amount of 0 or below is given, it does not subtract the money and instead prints Invalid withdrawal amount!
- Otherwise, subtract the money as normal.

```
def withdraw(self, amount: int):
def withdraw(self, amount: int):
    if amount <= 0:
        print('Invalid withdrawal amount!')
    else:
        self.balance = self.balance - amount
```

4. c:

/2

State one similarity and one difference between a class and a module in python.

- ☐ **Similarity:** Both are used to organise and group related code together, promoting reusability.
- ☐ **Difference:** A class is a blueprint for creating objects with attributes and methods (supports instantiation and OOP concepts like inheritance), whereas a module is a file containing Python code (functions, variables, classes) that can be imported — it is not instantiated.

Turn page for Question 5

Question 5 (11 marks):

/11

5.a: The program below does not execute successfully:

/3

```
1 import sys
2
3 def compute_discount(price, rate):
4     discounted = price * (1 - rate)
5     return discounted
6
7 item_price = float(sys.argv[1])
8 discount_rate = float(sys.argv[2])
9 print(price)
10 print(rate)
11 print(compute_discount(item_price, discount_rate))
```

When it is run, it produces the error shown:

```
Traceback (most recent call last):
  File exam_5a.py , line 9, in <module>
    print(price)
NameError: name 'price' is not defined
```

Answer the following:

(i) Why does the program fail to execute? (1)

price and rate are local parameters of the function compute_discount. They do not exist in the global (module) scope. Lines 9 and 10 try to print them outside the function, causing a NameError.

(ii) Fix the program by indicating the line number(s) you are changing and writing your updated code (you can add, remove, or modify lines as needed). (2)

```
# Change lines 9 and 10:
print(item_price) # was print(price)
print(discount_rate) # was print(rate)
```

5.b:

/8

Write a program called `converter.py` that takes n strings from the command-line arguments (excluding the filename). Each string represents a power operation in the format `value:unit` (e.g., `"100:C"` or `"212:F"`), and your program should compute and print the converted temperature for each valid input.

A string is considered a **valid temperature conversion** if it meets all of the following requirements:

- Contains :
- The value is on the left and can be converted to numbers.
- The unit is on the right and it can be either "C" or "F" (case sensitive)

Your program should:

- If the unit is C, convert to Fahrenheit: $F = C * 9/5 + 32$
- If the unit is F, convert to Celsius: $C = (F - 32) * 5/9$
- Ignore any string that is not a valid temperature conversion.

Example runs [user inputs are shown in **bold**]:

```
$ python3 converter.py 100:C abc 0:F 37:C
212.0
-17.78
98.6

$ python3 powers.py 32:F x y:z 25:C
0.0
77.0
```

Hint: Use the `sys` module, string method `split`, and exception handling (`try` and `except`).

```
import sys

args = sys.argv[1:]

if len(args) == 0:
    exit()

for arg in args:
    try:
        parts = arg.split(':')
        if len(parts) != 2:
            continue
        value = float(parts[0])
        unit = parts[1]
        if unit == 'C':
            result = value * 9 / 5 + 32
        elif unit == 'F':
            result = (value - 32) * 5 / 9
        else:
            continue
        print(round(result, 2))
    except ValueError:
        continue
```

Question 6 (13 marks):

/13

6.a:

/3

Mark the following statements true or false:

- (a) ☐ **TRUE** A `try` block must always be followed by an `except` block
- (b) ☐ **TRUE** When an unhandled exception occurs, the program stops running.
- (c) ☐ **FALSE** A `ValueError` is raised when a function receives an argument of the wrong type.
- (d) ☐ **FALSE** Opening a file in append mode ("`a`") will erase all existing contents.
- (e) ☐ **TRUE** The `len()` function can be used to find the number of characters in a string.
- (f) ☐ **FALSE** If a class does not define a `__str__` method, calling `str()` on its object will raise a `TypeError`.

6.b:

/10

Write a program called `count_words_odd_lines.py` that performs the following:

- Opens the file specified from the first command-line argument (excluding the program name). The program should not crash if this argument is missing.
- Reads the odd lines and counts the number of words in each of them.
- Prints the line followed by the number of words `<odd_line>:<#words>`.
- Ignores any additional command-line arguments.
- If any error occurs, print `'error'` and terminate the program.
- An empty file should also be treated as an error.

For example, to run the program with the file `essay.txt`:

```
$ python3 count_words_odd_lines.py essay.txt
```

Note: You have to use (`try` and `except`) to solve this question. Not using them will result in **zero marks** for the question. Make sure you handle and report *any possible exceptions* that might arise. Refer to the last page of the formula sheet if you need a reminder on file methods.

```
import sys

try:
    if len(sys.argv) < 2:
        raise Exception()
except:
    print('error')
    exit()

filename = sys.argv[1]
try:
    with open(filename, 'r') as f:
        lines = f.readlines()
except FileNotFoundError:
    print('error')
    exit()

try:
    if len(lines) == 0:
        raise Exception()
except:
    print('error')
    exit()

line_number = 1
for i in range(0, len(lines), 2):
    line = lines[i].strip()
    words = line.split()
    print(line + ':' + str(len(words)))
```

Question 7 (6 marks):

/6

Write a class called `Satellite` that models a satellite in an orbital simulation.

- Each satellite has a **name** (string), which must start with a letter and end with a number, a **battery** (integer, defaults to 100), and an **orbit_height** (float, defaults to 0.0).
- A **class variable** `total_satellites` that tracks the total number of created satellites.
- The `__init__` method must take in the name and an optional starting battery (default 100). Ensure the class variable is updated appropriately in the constructor. Raise a `ValueError` if the name does not start with a letter and end with a number.
- A **method** `boost(self, fuel_used)` that increases `orbit_height` by `fuel_used * 15.0` and decreases `battery` by `fuel_used`, only if `fuel_used > 0` and `fuel_used <= battery`.
- A **method** `recharge(self, amount)` that adds `amount` to `battery` if `amount > 0`.

```
class Satellite:
    total_satellites = 0

    def __init__(self, name: str, battery: int = 100):
        if not name[0].isalpha() or not name[-1].isdigit():
            raise ValueError('Name must start with a letter and end with a number.')
        self.name = name
        self.battery = battery
        self.orbit_height = 0.0
        Satellite.total_satellites += 1

    def boost(self, fuel_used):
        if fuel_used > 0 and fuel_used <= self.battery:
            self.orbit_height += fuel_used * 15.0
            self.battery -= fuel_used

    def recharge(self, amount):
        if amount > 0:
            self.battery += amount
```


Question 8 (20 marks):

/20

8.a:

/12

Write a function `is_palindrome(text)` that:

- Takes one parameter:
 - `text`: a non-empty string.
- Returns `True` if the string reads the same forwards and backwards, and `False` otherwise.

Example usages:

- `is_palindrome('racecar')` returns `True`
- `is_palindrome('a')` returns `True`
- `is_palindrome('abba')` returns `True`
- `is_palindrome('abcde')` returns `False`

Requirements:

Your solution must use:

- loops (for or while)
- string indexing (e.g. `text[i]`)

If you need, you may also want to use:

- `break`
- `continue`

Restrictions:You must **not** use:

- `in` to check membership (e.g. `if x in text`)
- slicing (e.g. `text[i:j]`)
- `set()`, `dict()`
- string methods of `count()`, `find()`, `index()`, `replace()`
- `re` module

```
def is_palindrome(text):
    length = len(text)
    i = 0
    while i < length // 2:
        if text[i] != text[length - 1 - i]:
            return False
        i += 1
    return True
```

8.b:

/8

Write a recursive function `flatten_words(lst)` that takes a list `lst` as parameters and returns a single string made by joining all words contained in the list, separated by a space.

The list may contain:

- Strings (words), and
- Lists of strings (but only one level of nesting)

You must solve the problem using recursion. Loops are not allowed.

Function specification:

- Function name: `flatten_words`
- Parameter: `lst` (a list containing strings or lists of strings)
- Returns: a string with all words joined by a single space
- If given an empty list, the function returns `''` (empty string)

Example usages:

- Calling `flatten_words(['I', 'Love', ['COMP', '9', '00', '1']])`, returns: 'I Love COMP 9 00 1'
- Calling `flatten_words(['Abd', 'Is'], 'The', ['Best', 'Tutor'])`, returns: 'Abd Is The Best Tutor'
- Calling `flatten_words(['I', ['Will', 'Fill'], 'USS', ['To', 'See', 'Armins', 'Cooking', 'Show']])`, returns: 'I Will Fill USS To See Armins Cooking Show'

Hint 1: To check if an item is a list or not, you may use: `if isinstance(item, list)`

Hint 2: You may find it helpful to use slicing to separate the first item in the list and the rest

Part A: Before writing any code, explain what the **base case** and **recursive case** are for this problem. (3 marks)

- (i) What is the base case for this recursion? (1)

The list `lst` is empty (`len(lst) == 0`). Return `''` (empty string) — there are no words left to join.

(ii) When should we recurse? What smaller subproblem should we recurse on? (2)

The list is non-empty. Look at the first element `lst[0]`:

- If it is a string, the subproblem is: combine `lst[0]` with the result of recursively calling `flatten_words(lst[1:])` (the rest of the list).
- If it is a list, the subproblem is: flatten that inner list first by recursively calling `flatten_words(lst[0])`, then combine that result with `flatten_words(lst[1:])`.
- In both cases we recurse on a strictly smaller list (`lst[1:]`), guaranteeing termination.

Part B: Now implement the function `flatten_words(lst)` making sure to include any necessary libraries. Your function **must** use recursion. Functions that are not recursive will receive **zero marks**. (5)

```
def flatten_words(lst):
    if len(lst) == 0:
        return ''

    first = lst[0]
    rest = flatten_words(lst[1:])

    if isinstance(first, list):
        first_str = flatten_words(first)
    else:
        first_str = first

    if first_str == '':
        return rest
    if rest == '':
        return first_str
    return first_str + ' ' + rest
```

Question 9 (10 marks):

/10

You have tasked a student with implementing 2 functions in `stats.py`. The docstrings for each function specify the correct behavior of the function (what the function is meant to do).

Below is the implementation of the functions that the student has provided. Your job is to develop test cases in `stats_test.py` to ensure that the student's implementation follows the requirements specified in the docstrings.

- For each function, develop 4 appropriate, non-overlapping test cases (you can define helper functions to help write your tests). (8)
- At least one of these test cases for each function above must either detect a mistake or something missed in the implementation that does not match the function's docstring. Clearly indicate at least one. (2)

You may use either Test Driver or Docstring Testing.

Hint: You may find it helpful to use `assert` statements, as well as exception handling (`try` and `except`). Refer to the formula sheet if you need more information on these.

stats.py

```
1  def running_max(values: list[int]) -> list[int] | None:
2      '''
3      Takes a list of integers and returns a list where each element
4      is the maximum value seen so far (running maximum).
5      Example: [3, 1, 4, 1, 5] -> [3, 3, 4, 4, 5]
6      Returns:
7      - List of running maxima if all elements are ints
8      - None if not a list, contains non-ints, or is empty
9      '''
10     if not isinstance(values, list) or not values:
11         return None
12     results = []
13     current_max = values[0]
14     for v in values:
15         if not isinstance(v, int):
16             return None
17         if v > current_max:
18             current_max = v
19         results.append(current_max)
20     return results
21
22
23  def flatten_dict(data: dict) -> dict | None:
24      '''
25      Returns a dict where keys are the original string keys and
26      values are the sum of each value list (list of ints).
27      Conditions:
28      - Keys must be strings, values must be lists of ints
29      - If a value list is empty, its sum is 0
30      - None if invalid or empty input
31      '''
32     if not isinstance(data, dict) or not data:
33         return None
34     result = {}
35     for k, v in data.items():
36         if not isinstance(k, str) or not isinstance(v, list):
```

```
37         return None
38     total = 0
39     for item in v:
40         if not isinstance(item, int):
41             return None
42         total += item
43     result[k] = total
44     return result
```

Write your answer here

```
import doctest
from stats import running_max, flatten_dict

# Tests for running_max
result = running_max([1, 2, 3, 4, 5])
assert result == [1, 2, 3, 4, 5], f'Expected [1,2,3,4,5], got {result}'

result = running_max([3, 1, 4, 1, 5])
assert result == [3, 3, 4, 4, 5], f'Expected [3,3,4,4,5], got {result}'

result = running_max([7])
assert result == [7], f'Expected [7], got {result}'

result = running_max([])
assert result is None, f'Expected None for empty list, got {result}'

# Tests for flatten_dict
def flatten_dict():
    """
    >>> flatten_dict({'a': [1, 2, 3], 'b': [4, 5]})
    {'a': 6, 'b': 9}

    >>> flatten_dict({'x': [], 'y': [1]})
    {'x': 0, 'y': 1}

    >>> flatten_dict([1, 2, 3]) is None
    True

    >>> flatten_dict({}) is None
    True
    """
```

Formula Sheet

Common types: int, float, bool, str**Common escape characters:** \n, \t, \', \", **Type information:** type(), isinstance(object, class)**Parsing methods:** int(), float(), str()**Common collections:** [] (list), () (tuple), {} (dictionary)**Collection classes:** list, dict, set, tuple**Common collection methods:** append, insert, remove, pop, index, reverse, copy**Keywords:**

def – Define a function e.g., def foo():

import – Import a module e.g., import sys

raise – Raise an exception e.g., raise Exception()

class – Define a class e.g., class Foo:

assert – Raises AssertionError if condition False e.g., assert x == 0, 'Failed!'

break, continue, return, yield

Common Modules:

sys – Provides access to command-line arguments (sys.argv).

Useful iterable Collection Operations:

sorted() – Returns a sorted list from an iterable.

reversed() – Returns a reverse iterator over an iterable.

list(iterable) – Returns a new list that contains all items from iterable, in the same order.

len() – Returns the length of the iterable.

next() – Returns the next item in the iterator, otherwise raises StopIteration.

iter() – Returns an iterator from the object passed.

Control Flow Syntax:

- if ... : ... [elif ... : ...] [else: ...]
- while ... : ... [else: ...]
- for ... in ... : ... [else: ...]
- try: ... except [...as ...]: ... [finally: ...]

Common Exceptions: Exception, SyntaxError, ValueError, TypeError, NameError, FileNotFoundError, IndexError, EOFError, KeyError, ZeroDivisionError, AssertionError

- **Assignment Operators:** = += *= -= /= //= %=
- **Arithmetic Operators:** + - * / // % **
- **Comparison Operators:** == != >= <= < >
- **Logical Operators:** not and or
- **Membership Operator:** in

Input/Output functions:

print() – Prints object(s) to standard output using their string representation.

input() – Reads a line of input from standard input (keyboard)

open(path, mode) – Opens file at path in given mode (read 'r', write 'w', append 'a') e.g., my file = open('a.txt', 'r')

File methods:

close() – Closes the file.

read(n) – Reads n characters from the file.

readline() – Reads a single line from the file.

readlines() – Reads all lines from the file as a list of strings.

write() – Writes a string to the file [does not add a newline (\n) automatically].

Dictionary - access:

d[key] – Returns the value indexed at key in d.

String, list, tuple - access:

seq[i] – Returns the element at index i of seq.

Slicing (returns a subsequence of seq):

- `seq[start:end]` – Returns elements from index start (inclusive) to end (exclusive).
- `seq[start:]` – Returns elements from index start to the end of seq.
- `seq[:end]` – Returns elements from the beginning of seq up to index end (exclusive).

String methods (str):

- `split(sep)` – Splits the string into a list of substrings using the specified separator sep. If sep is not provided, it defaults to whitespace e.g., `'A B'.split(' ')` returns `['A', 'B']`
- `lower()` / `upper()` – Returns a string in lowercase/uppercase.
- `isdigit()` – Returns True if all characters in the string are digits. Otherwise, False.
- `strip(chars)` – Returns a string with the specified chars removed from both the start and end of the string. If chars is not provided, it defaults to whitespace.
- `format()` / f-strings – Returns a formatted string using placeholders in `{}`.

Docstring testing:

```
def function_name(parameters):  
    """  
  
>>> function_name(arguments)  
    EXPECTED_OUTPUT  
    """
```

#IMPLEMENTATION HERE

```
import doctest  
result = doctest.testmod()  
n_tests_failed = result.failed
```

END OF EXAMINATION